

Jenkins challenges

Use the below code and implement our CI/CD stages.

- **Source Code:** <https://github.com/betawins/hiring-app.git>
 - **Repository contains the application code for the hiring app.**
- **Create one Declarative pipeline job**

Objective

To create a Jenkins Declarative Pipeline that automatically performs the CI/CD process for the Hiring application.

Implementation Steps

1. Log in to Jenkins.
2. From the Jenkins dashboard, click New Item.
3. Enter the job name:
hiring-app-dec
4. Select:
Pipeline
5. Click OK.
6. Scroll down to the Pipeline section.
7. For Definition, select:
Pipeline script
8. Enter the following Declarative Pipeline code:

```
pipeline {  
    agent any  
    tools {  
        maven 'MVN_HOME'  
    }  
    environment {  
        APP_NAME = 'HiringApp'  
        GIT_URL = 'https://github.com/betawins/hiring-app.git'
```

```
GIT_BRANCH = 'main'
SONAR_SERVER = 'sonarqube-server'
SONAR_URL = 'http://52.90.154.40:9002'
NEXUS_URL = '98.93.245.155:8081'
NEXUS_REPOSITORY = 'hiring_app'
NEXUS_CREDENTIALS = 'nexus-cred'
TOMCAT_URL = 'http://100.59.202.84:9090'
TOMCAT_CREDENTIALS = 'tomcat-cred'
TOMCAT_CONTEXT = 'hiring'
SLACK_CHANNEL = '#all-jenkins-bot'
SLACK_CREDENTIALS = 'slack-cred'
GROUP_ID = 'in.javahome'
ARTIFACT_ID = 'hiring'
APP_VERSION = '0.1'
WAR_FILE = 'target/hiring.war'
}
stages {
    stage('Git Clone') {
        steps {
            deleteDir()
            git(
                branch: "${GIT_BRANCH}",
                url: "${GIT_URL}"
            )
            sh '''
                echo "===== "
                echo "GIT CLONE"
                echo "===== "
                git branch --show-current
            '''
        }
    }
}
```

```

        git log -1 --oneline
        echo ""
        echo "Repository files:"
        ls -la
    ""
}
}
stage('Maven Build') {
    steps {
        sh ""
            echo "=====
            echo "MAVEN BUILD"
            echo "=====
            mvn -version
            mvn clean package -DskipTests
            echo ""
            echo "Generated files:"
            ls -lh target/
        ""
    }
}
stage('SonarQube Analysis') {
    steps {
        withSonarQubeEnv("${SONAR_SERVER}") {
            sh ""
                echo "=====
                echo "SONARQUBE ANALYSIS"
                echo "=====

```

```

        mvn \
        org.sonarsource.scanner.maven:sonar-maven-plugin:sonar \
        -Dsonar.projectKey=Test \
        -Dsonar.projectName=HiringApp \
        -Dsonar.sources=. \
        -Dsonar.host.url=$SONAR_URL
    ""
}
}
}
stage('Verify WAR') {
    steps {
        sh ""
        echo "=====
        echo "VERIFY WAR"
        echo "=====
        if [ ! -f "$WAR_FILE" ]; then
            echo "ERROR: hiring.war was not generated."
            ls -lah target/
            exit 1
        fi
        echo "WAR found:"
        ls -lh "$WAR_FILE"

        echo ""
        echo "WAR contents:"
        unzip -l "$WAR_FILE"
    ""
}

```

```

}

stage('Nexus Upload') {
    steps {
        nexusArtifactUploader(
            nexusVersion: 'nexus3',
            protocol: 'http',
            nexusUrl: "${NEXUS_URL}",
            groupId: "${GROUP_ID}",
            version: "${APP_VERSION}",
            repository: "${NEXUS_REPOSITORY}",
            credentialsId: "${NEXUS_CREDENTIALS}",
            artifacts: [
                [
                    artifactId: "${ARTIFACT_ID}",
                    classifier: "",
                    file: "${WAR_FILE}",
                    type: 'war'
                ]
            ]
        )
    }
}

stage('Deploy Tomcat') {
    steps {
        deploy(
            adapters: [
                tomcat9(
                    credentialsId: "${TOMCAT_CREDENTIALS}",
                    url: "${TOMCAT_URL}",

```

```

        path: "
    )
],
contextPath: "${TOMCAT_CONTEXT}",
war: "${WAR_FILE}"
)
}
}
stage('Slack Notification') {
    steps {
        slackSend(
            channel: "${SLACK_CHANNEL}",
            color: 'good',
            tokenCredentialId: "${SLACK_CREDENTIALS}",
            botUser: true,
            message: ""
Hiring App CI/CD SUCCESS
Application : ${APP_NAME}
Build      : #${BUILD_NUMBER}
Branch     : ${GIT_BRANCH}
SonarQube:
${SONAR_URL}
Nexus:
http://${NEXUS_URL}
Tomcat:
${TOMCAT_URL}/${TOMCAT_CONTEXT}/
Jenkins:
${BUILD_URL}
""

```

```

        )
    }
}
}
post {
    failure {
        slackSend(
            channel: "${SLACK_CHANNEL}",
            color: 'danger',
            tokenCredentialId: "${SLACK_CREDENTIALS}",
            botUser: true,
            message: ""
Hiring App CI/CD FAILED
Application : ${APP_NAME}
Build      : #${BUILD_NUMBER}
Branch     : ${GIT_BRANCH}
Please check Jenkins console output.
${BUILD_URL}
""
        )
    }
    success {
        echo "Hiring application deployed successfully."
        echo "Application URL:"
        echo "${TOMCAT_URL}/${TOMCAT_CONTEXT}/"
    }
}
}

```

9. Click Apply.

10. Click Save.
11. Click Build Now.
12. Open the build number.
13. Click Console Output.
14. Verify that the stages execute in the following order:

Git Clone



Maven Build



SonarQube Analysis



Verify WAR



Nexus Upload



Tomcat Deployment



Verify Tomcat

15. After successful deployment, access:

<http://100.59.202.84:9090/hiring/>

16. Verify that the Slack success notification is received.

hiring_app_dec Config - Jenkins hiring_app Config - Jenkins Tools - Manage Jenkins - Jenkins HTTP Status 404 - Not Found

Not secure 3.93.154.157:8080/job/hiring_app_dec/configure

Jenkins / hiring_app_dec / Configure

Configure

Pipeline script

General Triggers Pipeline Advanced

Script ?

```
1 pipeline {
2
3   agent any
4
5   tools {
6     maven 'MVN_HOME'
7   }
8
9   environment {
10
11     APP_NAME = 'HiringApp'
12
13     GIT_URL = 'https://github.com/betawins/hiring-app.git'
14     GIT_BRANCH = 'main'
15 }
```

Save Apply

31° 12:39 06-10-2026

hiring_app_dec #1 - Jenkins hiring_app Config - Jenkins Tools - Manage Jenkins - Jenkins HTTP Status 404 - Not Found

Not secure 3.93.154.157:8080/job/hiring_app_dec/1/

Jenkins / hiring_app_dec / #1

Status #1 (Oct 6, 2026, 7:05:21 AM) Add description Keep this build forever

Changes

Console Output

Delete build '#1'

Git Build Data

Pipeline Overview

Edit build information

Restart from Stage

Replay

Pipeline Steps

Workspaces

Started by user vamshikumar

Started 24 sec ago Took 17 sec

git Revision: 9405b107fdffdf68e69fe953ea1f9669784f9d6a Repository: <https://github.com/betawins/hiring-app.git>

- refs/remotes/origin/main

</> No changes.

REST API Jenkins 2.580.1

31° 12:38 06-10-2026

hiring_app_dec #1 Console - Jenkins hiring_app Config - Jenkins Tools - Manage Jenkins - Jenkins HTTP Status 404 - Not Found

Not secure 3.93.154.157:8080/job/hiring_app_dec/1/console

Jenkins / hiring_app_dec / #1

```
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Hiring application deployed successfully.
[Pipeline] echo
Application URL:
[Pipeline] echo
http://100.59.202.84:9090/hiring/
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Jenkins 2.580.1

Not secure 52.90.154.40:9002/projects

Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine.

There's a new version of SonarQube Server available. Upgrade to SonarQube Server and get access to enterprise features. [Learn More](#)

The way in which security, reliability, and maintainability counts and ratings are calculated has changed. [Learn more in SonarQube documentation](#)

SonarQube community

Projects Issues Rules Quality profiles Quality gates Administration More

My Favorites All

Filters

Quality gate

Passed 3

Failed 0

Security

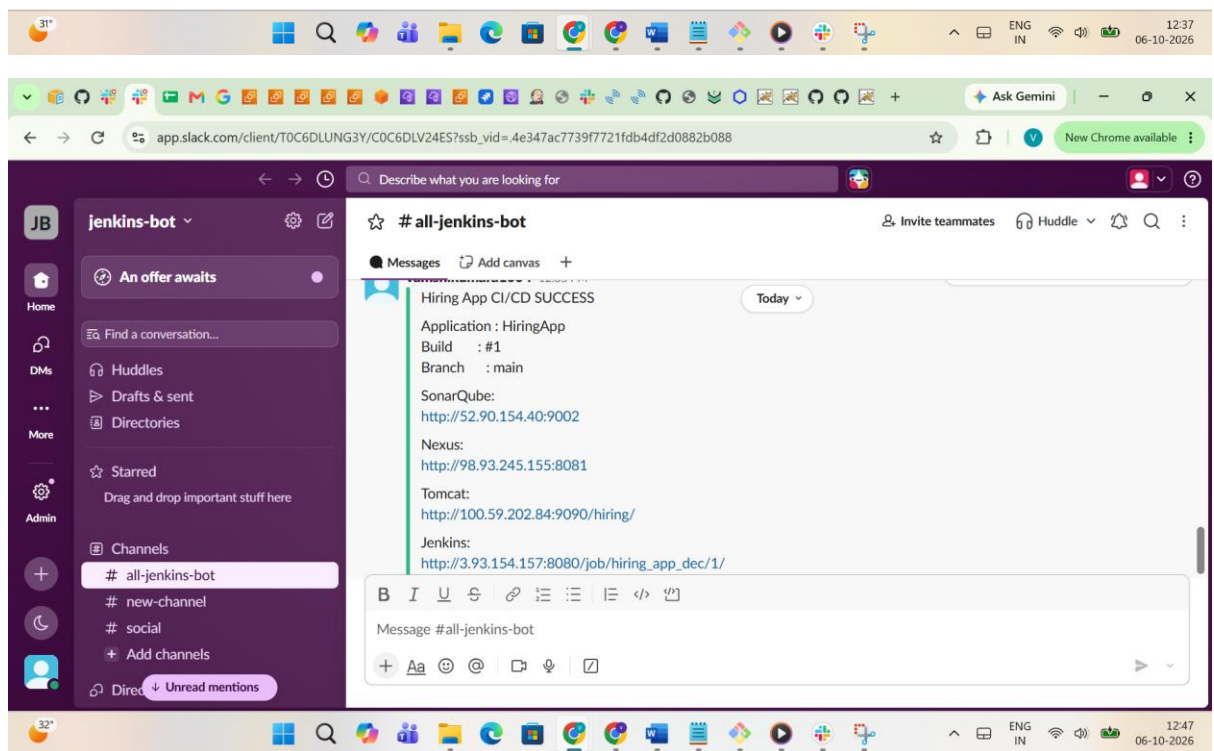
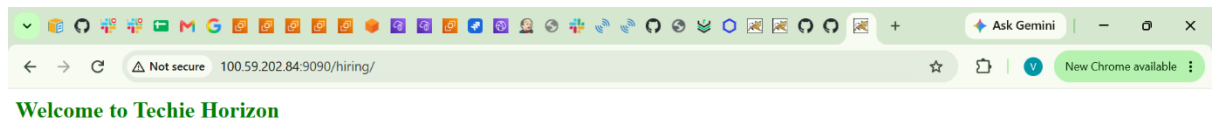
Search projects Perspective Overall Status Sort by Name 3 project(s)

Create Project

HiringApp Public Passed

Last analysis: 1 hour ago - 45 Lines of Code - XML, Docker, ...

B 2	C 3	A 0	A —	—	0.0%
Security	Reliability	Maintainability	Hotspots Reviewed	Coverage	Duplications



- **Create one Scripted pipeline job**

Objective

To implement the same CI/CD process using Scripted Pipeline syntax.

Scripted Pipeline uses a node {} block and Groovy-based flow control, whereas Declarative Pipeline uses the pipeline {} structure.

Implementation Steps

1. Go to:

Jenkins → New Item

2. Enter:

hiring-app-scri

3. Select:

Pipeline

4. Click OK.

5. Scroll to the Pipeline section.

6. Select:

Definition: Pipeline script

7. Enter:

```
node {  
    // =====  
    // MAVEN  
    // =====  
    def mvnHome = tool(  
        name: 'MVN_HOME',  
        type: 'maven'  
    )  
    // =====  
    // APPLICATION  
    // =====  
    def APP_NAME = 'HiringApp'  
    // =====  
    // GIT  
    // =====  
    def GIT_URL = 'https://github.com/betawins/hiring-app.git'  
    def GIT_BRANCH = 'main'  
    // =====  
    // SONARQUBE
```

```
// =====
def SONAR_SERVER = 'sonarqube-server'
def SONAR_URL = 'http://52.90.154.40:9002'
// =====
// NEXUS
// =====
def NEXUS_URL = '98.93.245.155:8081'
def NEXUS_REPOSITORY = 'hiring_app'
def NEXUS_CREDENTIALS = 'nexus-cred'
// =====
// MAVEN ARTIFACT
// =====
def GROUP_ID = 'in.javahome'
def ARTIFACT_ID = 'hiring'
def VERSION = '0.1'
def WAR_FILE = 'target/hiring.war'
// =====
// TOMCAT
// =====
def TOMCAT_URL = 'http://100.59.202.84:9090'
def TOMCAT_CREDENTIALS = 'tomcat-cred'
def TOMCAT_CONTEXT = 'hiring'
// =====
// SLACK
// =====
def SLACK_CHANNEL = '#all-jenkins-bot'
def SLACK_CREDENTIALS = 'slack-cred'
try {
    // =====
```

```

// MAVEN PATH

// =====

env.PATH = "${mvnHome}/bin:${env.PATH}"

// =====

// STAGE 1 - GIT CLONE

// =====

stage('Git Clone') {

    echo '===== '

    echo 'STAGE 1: GIT CLONE'

    echo '===== '

    deleteDir()

    git(

        branch: GIT_BRANCH,

        url: GIT_URL

    )

    sh '''

        echo "Current branch:"

        git branch --show-current

        echo ""

        echo "Latest commit:"

        git log -1 --oneline

        echo ""

        echo "Repository files:"

        ls -la

    '''

}

// =====

// STAGE 2 - MAVEN BUILD

// =====

```

```

stage('Maven Build') {
    echo '=====
    echo 'STAGE 2: MAVEN BUILD'
    echo '=====
    sh '''
        echo "Maven version:"
        mvn --version
        echo ""
        echo "Java version:"
        java -version
        echo ""
        echo "Running Maven build..."
        mvn clean package -DskipTests
        echo ""
        echo "Target directory:"
        ls -lh target/
    '''
}

// =====
// STAGE 3 - SONARQUBE
// =====

stage('SonarQube Analysis') {
    echo '=====
    echo 'STAGE 3: SONARQUBE ANALYSIS'
    echo '=====
    withSonarQubeEnv(SONAR_SERVER) {
        sh """
            mvn \
            org.sonarsource.scanner.maven:sonar-maven-plugin:sonar \

```

```

        -Dsonar.projectKey=hiring-app \
        -Dsonar.projectName=HiringApp \
        -Dsonar.projectVersion=${env.BUILD_NUMBER} \
        -Dsonar.host.url=${SONAR_URL}
    """"

}

echo 'SonarQube analysis completed successfully.'

}

// =====
// STAGE 4 - VERIFY WAR
// =====

stage('Verify WAR') {

    echo '=====
    echo 'STAGE 4: VERIFY WAR'
    echo '=====
    sh """"

    if [ ! -f "${WAR_FILE}" ]; then

        echo "ERROR: WAR file not found."

        echo ""

        echo "Target directory:"

        ls -lh target/

        exit 1

    fi

    echo "WAR found successfully:"

    ls -lh "${WAR_FILE}"

    echo ""

    echo "WAR contents:"

    unzip -l "${WAR_FILE}" | head -30

    """"

```

```

}

// =====
// STAGE 5 - NEXUS UPLOAD
// =====

stage('Nexus Upload') {
    echo '=====
    echo 'STAGE 5: NEXUS UPLOAD'
    echo '=====
    echo "Nexus URL: ${NEXUS_URL}"
    echo "Repository: ${NEXUS_REPOSITORY}"
    echo "Artifact: ${WAR_FILE}"
    nexusArtifactUploader(
        nexusVersion: 'nexus3',
        protocol: 'http',
        nexusUrl: NEXUS_URL,
        groupId: GROUP_ID,
        version: VERSION,
        repository: NEXUS_REPOSITORY,
        credentialsId: NEXUS_CREDENTIALS,
        artifacts: [
            [
                artifactId: ARTIFACT_ID,
                classifier: "",
                file: WAR_FILE,
                type: 'war'
            ]
        ]
    )
    echo 'Nexus upload completed successfully.'
}

```

```

}

// =====

// STAGE 6 - TOMCAT DEPLOYMENT

// =====

stage('Deploy to Tomcat') {
    echo '=====
    echo 'STAGE 6: TOMCAT DEPLOYMENT'
    echo '=====
    echo "Tomcat URL: ${TOMCAT_URL}"
    echo "WAR file: ${WAR_FILE}"
    echo "Context path: /${TOMCAT_CONTEXT}"
    deploy(
        adapters: [
            tomcat9(
                credentialsId: TOMCAT_CREDENTIALS,
                url: TOMCAT_URL,
                path: "
            )
        ],
        contextPath: TOMCAT_CONTEXT,
        war: WAR_FILE
    )
    echo 'Tomcat deployment completed successfully.'
}

// =====

// STAGE 7 - VERIFY TOMCAT

// =====

stage('Verify Tomcat Deployment') {
    echo '=====

```

```

echo 'STAGE 7: VERIFY TOMCAT DEPLOYMENT'

echo '===== '

sh ""

    set +e

    echo "Checking Tomcat application..."

    HTTP_STATUS=$(curl -s -o /dev/null -w "%{http_code}" \
    "${TOMCAT_URL}/${TOMCAT_CONTEXT}/")

    echo ""

    echo "Application URL:"

    echo "${TOMCAT_URL}/${TOMCAT_CONTEXT}/"

    echo ""

    echo "HTTP Status:"

    echo "\${HTTP_STATUS}"

    echo ""

    if [ "\${HTTP_STATUS}" = "200" ]; then

        echo "Hiring application is running successfully."

    elif [ "\${HTTP_STATUS}" = "302" ]; then

        echo "Hiring application is deployed and returned HTTP 302."

    elif [ "\${HTTP_STATUS}" = "404" ]; then

        echo "HTTP 404 received."

        echo "The application may be deployed but does not have"

        echo "a valid welcome page at this path."

    else

        echo "Application returned HTTP status \${HTTP_STATUS}."

    fi

    echo ""

    echo "Tomcat verification completed."

    ""

}

```

```

// =====
// STAGE 8 - SLACK NOTIFICATION
// =====

stage('Slack Notification') {

    echo '===== '
    echo 'STAGE 8: SLACK NOTIFICATION'
    echo '===== '

    slackSend(

        channel: SLACK_CHANNEL,

        color: 'good',

        tokenCredentialId: SLACK_CREDENTIALS,

        botUser: true,

        message: ""

:large_green_circle: *HIRING APP PIPELINE SUCCESS*

Application : ${APP_NAME}

Job      : ${env.JOB_NAME}

Build    : #${env.BUILD_NUMBER}

Branch   : ${GIT_BRANCH}

Git:

${GIT_URL}

SonarQube:

${SONAR_URL}

Nexus:

http://${NEXUS_URL}

Repository:

${NEXUS_REPOSITORY}

WAR:

${WAR_FILE}

Tomcat:

```

`${TOMCAT_URL}/${TOMCAT_CONTEXT}/`

Jenkins:

`${env.BUILD_URL}`

""""

```
    )
    echo 'Slack success notification sent.'
}

// =====
// PIPELINE SUCCESS
// =====

echo '=====
echo 'SCRIPTED PIPELINE SUCCESS'
echo '=====
echo "Application: ${APP_NAME}"
echo "Build: #${env.BUILD_NUMBER}"
echo "Nexus Repository: ${NEXUS_REPOSITORY}"
echo "WAR: ${WAR_FILE}"
echo "Tomcat URL:"
echo "${TOMCAT_URL}/${TOMCAT_CONTEXT}/"
}

// =====
// FAILURE
// =====

catch (err) {
    echo '=====
    echo 'SCRIPTED PIPELINE FAILED'
    echo '=====
    echo "Error: ${err}"
    // =====
```

```

// SLACK FAILURE NOTIFICATION

// =====

slackSend(
    channel: SLACK_CHANNEL,
    color: 'danger',
    tokenCredentialId: SLACK_CREDENTIALS,
    botUser: true,
    message: ""

:red_circle: *HIRING APP PIPELINE FAILED*

Application : ${APP_NAME}
Job      : ${env.JOB_NAME}
Build    : #${env.BUILD_NUMBER}
Branch   : ${GIT_BRANCH}

Error:

${err}

Please check Jenkins Console Output.

Jenkins:

${env.BUILD_URL}

""

    )

    echo 'Slack failure notification sent.'

    throw err

}

// =====

// FINALLY

// =====

finally {

    echo '===== '

    echo 'PIPELINE EXECUTION COMPLETED'

```

```
    echo '====='  
  }  
}
```

8. Click Apply.

9. Click Save.

10. Click Build Now.

11. Open Console Output.

12. Verify the CI/CD sequence:

Git Clone



Maven Build



SonarQube



WAR Verification



Nexus



Tomcat



Slack

hiring_app_scri Config - Jenkins hiring_app Config - Jenkins Tools - Manage Jenkins - Jenkins HTTP Status 404 - Not Found

Not secure 3.93.154.157:8080/job/hiring_app_scri/configure

Jenkins / hiring_app_scri / Configure

Configure

- General
- Triggers
- Pipeline
- Advanced

Pipeline script

Script ?

```
1 node {
2
3   def mvnHome = tool(
4     name: 'MVN_HOME',
5     type: 'maven'
6   )
7
8   def GIT_URL = 'https://github.com/betawins/hiring-app.git'
9   def GIT_BRANCH = 'main'
10
11  def SONAR_SERVER = 'sonarqube-server'
12  def SONAR_URL = 'http://52.90.154.40:9002'
13
14  def NEXUS_URL = '98.93.245.155:8081'
15  def NEXUS_REPOSITORY = 'hiring_app'
```

Save Apply

31°

Windows taskbar icons: File Explorer, Edge, VS Code, etc.

System tray: ENG IN, 12:44, 06-10-2026

hiring_app_scri #1 - Jenkins hiring_app Config - Jenkins Tools - Manage Jenkins - Jenkins HTTP Status 404 - Not Found

Not secure 3.93.154.157:8080/job/hiring_app_scri/1/

Jenkins / hiring_app_scri / #1

Status #1 (Oct 6, 2026, 7:13:31AM)

Add description Keep this build forever

Changes

Console Output

Delete build '#1'

Git Build Data

Pipeline Overview

Edit build information

Replay

Pipeline Steps

Workspaces

Started by user [vamshikumar](#)

Started 1 min 55 sec ago
Took 15 sec

git

Revision: 9405b107fd6f68e69fe953ea1f9669784f9d6a
Repository: <https://github.com/betawins/hiring-app.git>

- refs/remotes/origin/main

No changes.

REST API Jenkins 2.580.1

32°

Windows taskbar icons: File Explorer, Edge, VS Code, etc.

System tray: ENG IN, 12:45, 06-10-2026

hiring_app_scri #1 Console - Jenkins | hiring_app Config - Jenkins | Tools - Manage Jenkins - Jenkins | HTTP Status 404 - Not Found

Not secure 3.93.154.157:8080/job/hiring_app_scri/1/console

Jenkins / hiring_app_scri / #1

```
Uploading: http://98.93.245.155:8081/repository/hiring_app/in/javahome/hiring/0.1/hiring-0.1.war
100 % completed (1.7 kB / 1.7 kB).
Uploaded: http://98.93.245.155:8081/repository/hiring_app/in/javahome/hiring/0.1/hiring-0.1.war (1.7 kB at 39
kB/s)
Uploading artifact hiring.war completed.
[Pipeline] echo
Nexus upload completed successfully.
[Pipeline] }
[Pipeline] // stage
[Pipeline] echo
=====
[Pipeline] echo
SCRIPTED PIPELINE SUCCESS
[Pipeline] echo
=====
[Pipeline] echo
Pipeline execution completed.
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

32° | Windows Taskbar | 12:45 06-10-2026

Not secure 52.90.154.40:9002/projects

Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine.

- There's a new version of SonarQube Server available. Upgrade to SonarQube Server and get access to enterprise features. [Learn More](#)
- The way in which security, reliability, and maintainability counts and ratings are calculated has changed. [Learn more in SonarQube documentation](#)

SonarQube community

Projects | Issues | Rules | Quality profiles | Quality gates | Administration | More

My Favorites | All

Filters

Quality gate

- Passed 3
- Failed 0

Security

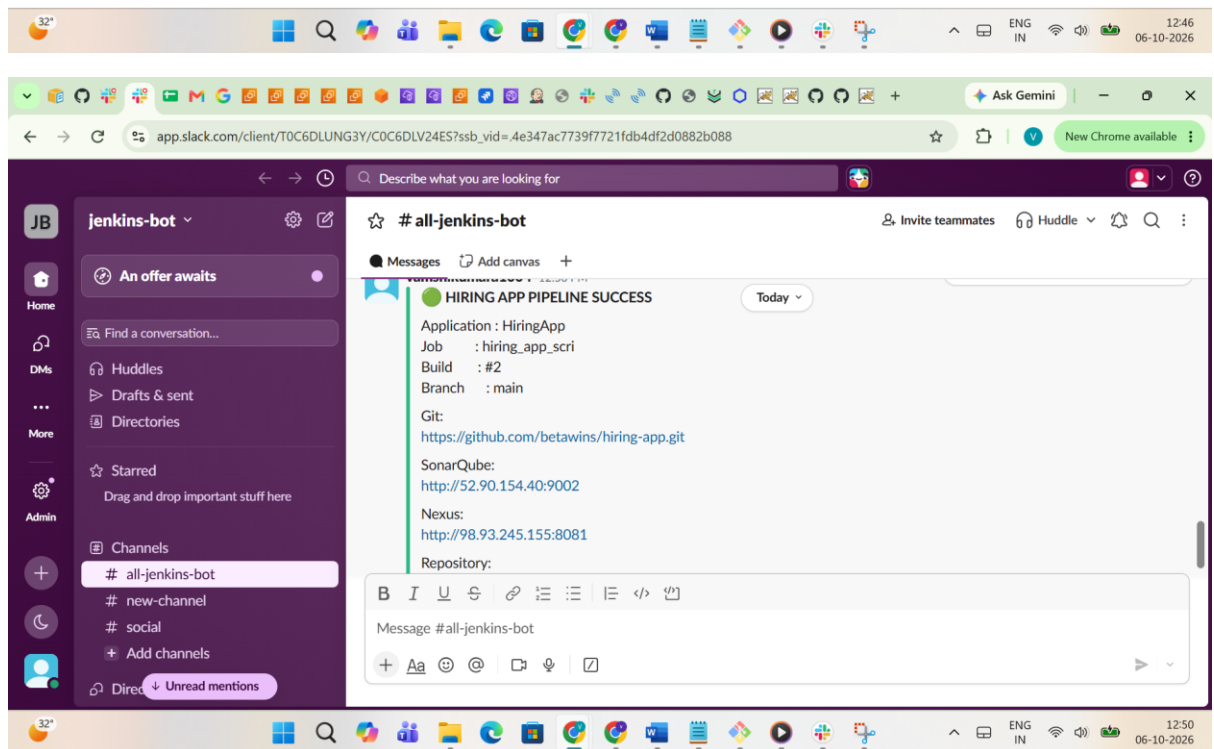
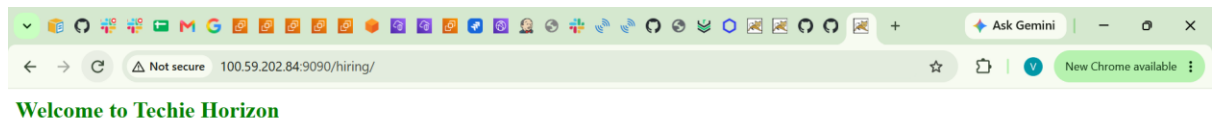
Create Project

Search projects | Perspective | Overall Status | Sort by Name | 3 project(s)

HiringApp (Public) Passed

Last analysis: 1 hour ago - 45 Lines of Code - XML, Docker, ...

B 2	C 3	A 0	A —	—	0.0%
Security	Reliability	Maintainability	Hotspots Reviewed	Coverage	Duplications



- **Create one multi stage pipeline job**

Objective

To create a Jenkins Pipeline containing multiple sequential stages for the Hiring application.

A Jenkins stage represents a logically distinct part of the delivery process such as Build, Test, or Deploy.

Implementation Steps

1. Go to:

Jenkins → New Item

2. Enter:

hiring-app-multi

3. Select:

Pipeline

4. Click **OK**.

5. Scroll to the **Pipeline** section.

6. Select:

Definition: Pipeline script

7. Enter:

```
pipeline {
```

```
    agent any
```

```
    tools {
```

```
        // Jenkins → Manage Jenkins → Tools
```

```
        // Maven installation name
```

```
        maven 'MVN_HOME'
```

```
    }
```

```
    environment {
```

```
        // =====
```

```
        // APPLICATION
```

```
// =====
```

```
APP_NAME = 'HiringApp'
```

```
// =====
```

```
// GIT
```

```
// =====
```

```
GIT_URL = 'https://github.com/betawins/hiring-app.git'
```

```
GIT_BRANCH = 'main'
```

```
// =====
```

```
// SONARQUBE
```

```
// =====
```

```
SONAR_SERVER = 'sonarqube-server'
```

```
SONAR_URL = 'http://52.90.154.40:9002'
```

```
// =====
```

```
// NEXUS
```

```
// =====
```

```
NEXUS_URL = '98.93.245.155:8081'
```

```
NEXUS_REPOSITORY = 'hiring_app'
```

```
NEXUS_CREDENTIALS = 'nexus-cred'
```

```
GROUP_ID = 'in.javahome'
```

```
ARTIFACT_ID = 'hiring'
```

```
VERSION = '0.1'
```

```
// =====  
  
// TOMCAT  
  
// =====  
  
TOMCAT_URL = 'http://100.59.202.84:9090'  
TOMCAT_CREDENTIALS = 'tomcat-cred'  
TOMCAT_CONTEXT = 'hiring'  
  
// =====  
  
// SLACK  
  
// =====  
  
SLACK_CHANNEL = '#all-jenkins-bot'  
SLACK_CREDENTIALS = 'slack-cred'  
  
// =====  
  
// WAR  
  
// =====  
  
WAR_FILE = 'target/hiring.war'  
}  
  
stages {  
  
// =====  
  
// STAGE 1 - CHECKOUT  
  
// =====
```

```
stage('Git Clone') {

    steps {

        echo '===== '
        echo 'STAGE 1: GIT CLONE'
        echo '===== '

        git(
            branch: "${GIT_BRANCH}",
            url: "${GIT_URL}"
        )

        sh '''
            echo "Current branch:"
            git branch --show-current

            echo ""
            echo "Latest commit:"
            git log -1 --oneline

            echo ""
            echo "Repository files:"
            ls -la
        '''
    }
}
```

```
// =====  
// STAGE 2 - MAVEN BUILD  
// =====
```

```
stage('Maven Build') {
```

```
  steps {
```

```
    echo '====='
```

```
    echo 'STAGE 2: MAVEN BUILD'
```

```
    echo '====='
```

```
    sh '''
```

```
      set -e
```

```
      echo "Maven version:"
```

```
      mvn --version
```

```
      echo ""
```

```
      echo "Java version:"
```

```
      java -version
```

```
      echo ""
```

```
      echo "Running Maven build..."
```

```
      mvn clean package -DskipTests
```

```
      echo ""
```

```
      echo "Build completed successfully."
```

```

        echo ""
        echo "Target directory:"
        ls -lh target/
    ""
}
}

```

```

// =====
// STAGE 3 - SONARQUBE
// =====

```

```

stage('SonarQube Analysis') {

```

```

    steps {

```

```

        echo '=====
        echo 'STAGE 3: SONARQUBE ANALYSIS'
        echo '=====

```

```

        withSonarQubeEnv("${SONAR_SERVER}") {

```

```

            sh ""

```

```

                set -e

```

```

                echo "SonarQube URL:"

```

```

                echo "${SONAR_URL}"

```

```

    echo ""
    echo "Starting SonarQube analysis..."

    mvn \
    org.sonarsource.scanner.maven:sonar-maven-plugin:5.8.0.7211:sonar \
    -Dsonar.projectKey=hiring-app \
    -Dsonar.projectName=HiringApp \
    -Dsonar.projectVersion=${BUILD_NUMBER} \
    -Dsonar.host.url=${SONAR_URL}

    echo ""
    echo "SonarQube analysis completed successfully."
  ""
}
}
}

// =====
// STAGE 4 - VERIFY WAR
// =====

stage('Verify WAR') {

  steps {

    echo '=====
    echo 'STAGE 4: VERIFY WAR'
    echo '=====

```

```
sh ""

set -e

echo "Expected WAR:"
echo "${WAR_FILE}"

if [ ! -f "${WAR_FILE}" ]; then

    echo ""
    echo "ERROR: WAR file not found."

    echo ""
    echo "Contents of target:"
    ls -lah target/

    exit 1
fi

echo ""
echo "WAR file found successfully."

ls -lh "${WAR_FILE}"

echo ""
echo "WAR contents:"

unzip -l "${WAR_FILE}" | head -40

""
```

```
}  
}
```

```
// =====  
// STAGE 5 - NEXUS UPLOAD  
// =====
```

```
stage('Nexus Upload') {
```

```
  steps {
```

```
    echo '====='
```

```
    echo 'STAGE 5: NEXUS UPLOAD'
```

```
    echo '====='
```

```
    nexusArtifactUploader(  
      nexusVersion: 'nexus3',  
      protocol: 'http',  
      nexusUrl: "${NEXUS_URL}",  
      groupId: "${GROUP_ID}",  
      version: "${VERSION}",  
      repository: "${NEXUS_REPOSITORY}",  
    )  
  }  
}
```

```

        credentialsId: "${NEXUS_CREDENTIALS}",

        artifacts: [

            [
                artifactId: "${ARTIFACT_ID}",
                classifier: "",
                file: "${WAR_FILE}",
                type: 'war'
            ]
        ]
    )

    echo 'Nexus upload completed successfully.'
}

}

// =====
// STAGE 6 - TOMCAT DEPLOYMENT
// =====

stage('Deploy to Tomcat') {

    steps {

        echo '=====
echo 'STAGE 6: TOMCAT DEPLOYMENT'

```

```

echo '=====
echo "Tomcat URL: ${TOMCAT_URL}"
echo "WAR file: ${WAR_FILE}"
echo "Context: ${TOMCAT_CONTEXT}"
deploy(
  adapters: [
    tomcat9(
      credentialsId: "${TOMCAT_CREDENTIALS}",
      url: "${TOMCAT_URL}",
      path: "
    )
  ],
  contextPath: "${TOMCAT_CONTEXT}",
  war: "${WAR_FILE}"
)
echo ""
echo "Tomcat deployment completed successfully."
}
}
// =====
// STAGE 7 - VERIFY TOMCAT
// =====
stage('Verify Tomcat') {
  steps {
    echo '=====
    echo 'STAGE 7: VERIFY TOMCAT'
    echo '=====
    sh ""
    set +e

```

```

    echo "Checking deployed application..."

    HTTP_STATUS=$(curl -s -o /dev/null -w "%{http_code}" \
    "${TOMCAT_URL}/${TOMCAT_CONTEXT}/")

    echo ""

    echo "Application URL:"

    echo "${TOMCAT_URL}/${TOMCAT_CONTEXT}/"

    echo ""

    echo "HTTP Status: ${HTTP_STATUS}"

    if [ "${HTTP_STATUS}" = "200" ]; then

        echo "Application is responding successfully."

    elif [ "${HTTP_STATUS}" = "302" ]; then

        echo "Application is deployed and returned HTTP 302."

    elif [ "${HTTP_STATUS}" = "404" ]; then

        echo "Application returned HTTP 404."

        echo "Check the WAR welcome page or application URL."

    else

        echo "Application returned HTTP ${HTTP_STATUS}."

    fi

    ""

}

}

// =====

// STAGE 8 - SLACK NOTIFICATION

// =====

stage('Slack Notification') {

    steps {

        echo '=====

        echo 'STAGE 8: SLACK NOTIFICATION'

        echo '=====

```

```

        slackSend(
            channel: "${SLACK_CHANNEL}",
            color: 'good',
            tokenCredentialId: "${SLACK_CREDENTIALS}",
            botUser: true,
            message: ""

PIPELINE SUCCESS

Application : ${APP_NAME}

Job      : ${JOB_NAME}

Build    : #${BUILD_NUMBER}

Branch   : ${GIT_BRANCH}

SonarQube:

${SONAR_URL}

Nexus:

http://${NEXUS_URL}

Repository:

${NEXUS_REPOSITORY}

WAR:

${WAR_FILE}

Tomcat:

${TOMCAT_URL}/${TOMCAT_CONTEXT}/

Jenkins:

${BUILD_URL}

""

    )

}

}

}

// =====

```

```

// POST ACTIONS

// =====

post {
    success {
        echo '=====
        echo 'MULTI-STAGE PIPELINE SUCCESS'
        echo '=====
        echo "Application deployed successfully."
        echo "Application URL:"
        echo "${TOMCAT_URL}/${TOMCAT_CONTEXT}/"
    }
    failure {
        echo '=====
        echo 'MULTI-STAGE PIPELINE FAILED'
        echo '=====
        slackSend(
            channel: "${SLACK_CHANNEL}",
            color: 'danger',
            tokenCredentialId: "${SLACK_CREDENTIALS}",
            botUser: true,
            message: ""
PIPELINE FAILED
Application : ${APP_NAME}
Job      : ${JOB_NAME}
Build    : #${BUILD_NUMBER}
Branch   : ${GIT_BRANCH}
Please check Jenkins Console Output.
Jenkins:
${BUILD_URL}

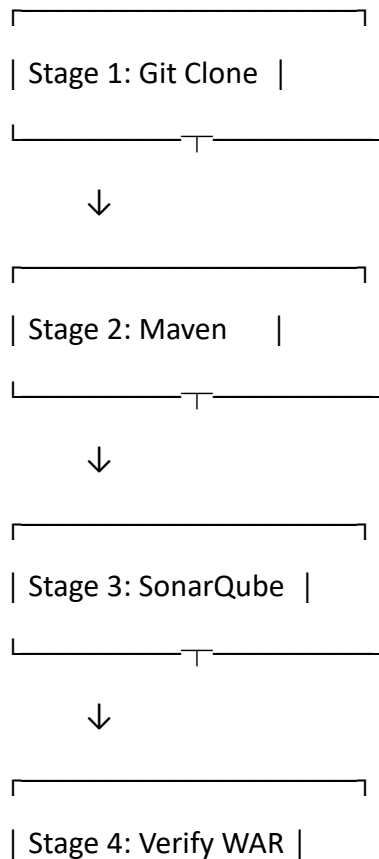
```

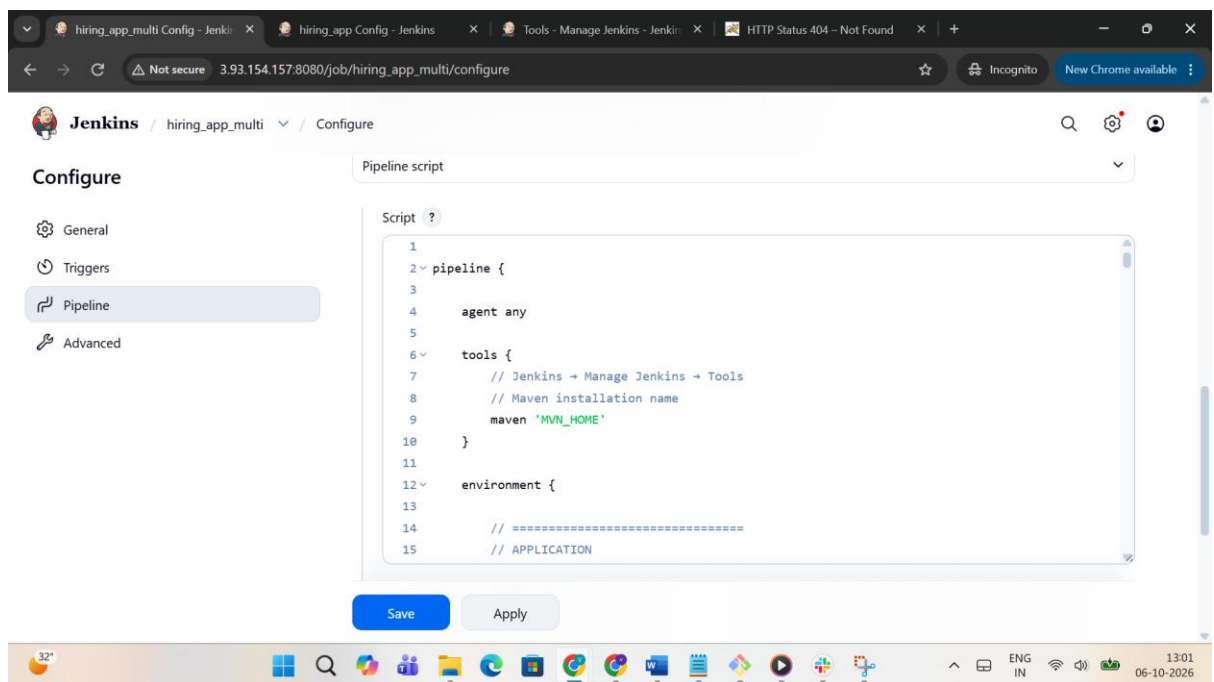
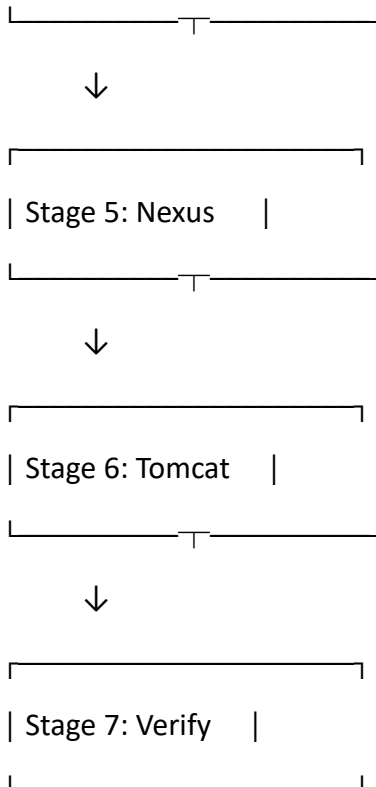
====

```
    )  
  }  
  always {  
    echo '====='  
    echo 'PIPELINE EXECUTION COMPLETED'  
    echo '====='  
  }  
}  
}
```

8. Click **Apply**.
9. Click **Save**.
10. Click **Build Now**.
11. Jenkins will display the stages separately in the Pipeline visualization.

The expected flow is:





hiring_app_multi #1 - Jenkins hiring_app Config - Jenkins Tools - Manage Jenkins - Jenkins HTTP Status 404 - Not Found

Not secure 3.93.154.157:8080/job/hiring_app_multi/1/ Incognito New Chrome available

Jenkins / hiring_app_multi / #1

Status #1 (Oct 6, 2026, 7:29:22 AM) Add description Keep this build forever

Changes

Console Output

Delete build '#1'

Git Build Data

Pipeline Overview

Edit build information

Restart from Stage

Replay

Pipeline Steps

Workspaces

Started by user vamshikumar

Revision: 9405b107fdffd68e69fe953ea1f9669784f9d6a

Repository: <https://github.com/betawins/hiring-app.git>

- refs/remotes/origin/main

No changes.

Started 30 sec ago

Took 18 sec

REST API Jenkins 2.580.1

hiring_app_multi #1 Console - hiring_app Config - Jenkins Tools - Manage Jenkins - Jenkins HTTP Status 404 - Not Found

Not secure 3.93.154.157:8080/job/hiring_app_multi/1/console Incognito New Chrome available

Jenkins / hiring_app_multi / #1

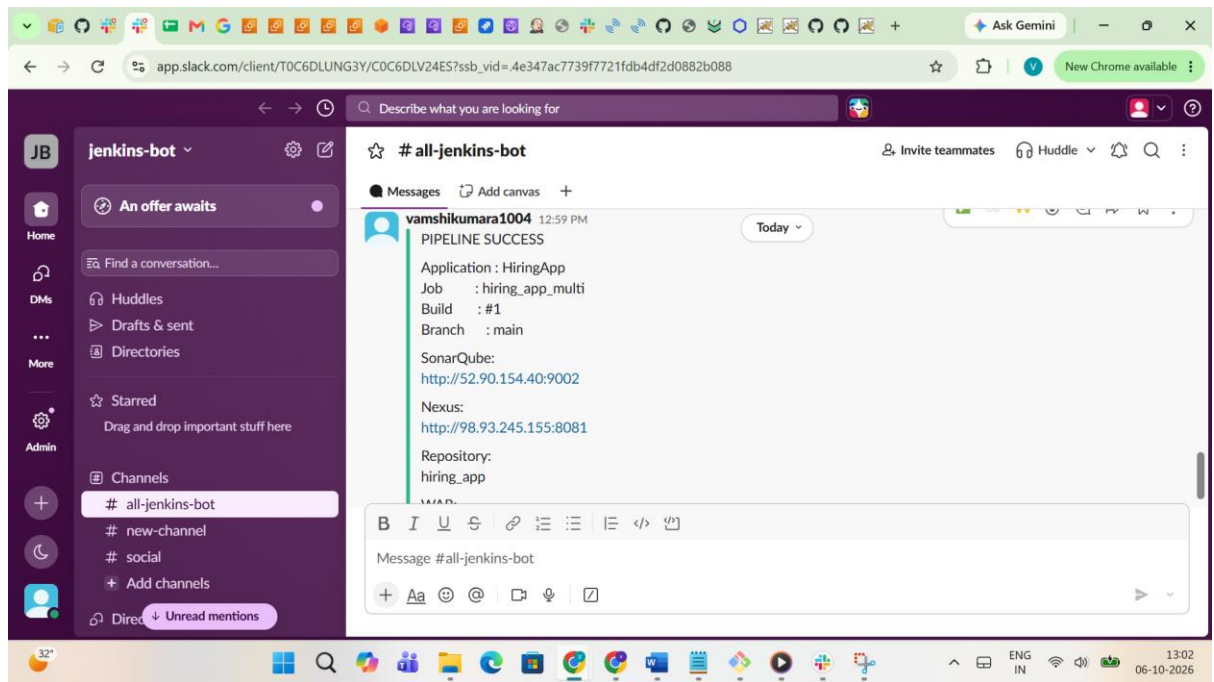
```
=====  
[Pipeline] echo  
Application deployed successfully.  
[Pipeline] echo  
Application URL:  
[Pipeline] echo  
http://100.59.202.84:9090/hiring/  
[Pipeline] }  
[Pipeline] // stage  
[Pipeline] }  
[Pipeline] // withEnv  
[Pipeline] }  
[Pipeline] // withEnv  
[Pipeline] }  
[Pipeline] // node  
[Pipeline] End of Pipeline  
Finished: SUCCESS
```

Jenkins 2.580.1

Welcome to Techie Horizon

32°

^ ENG 13:00
IN 06-10-2026



- **Create one parallel stage pipeline job**

Objective

To create a Jenkins Pipeline where selected activities execute **in parallel** instead of sequentially.

Declarative Pipeline supports a parallel section containing stages that can run concurrently.

Implementation Steps

1. Go to:

Jenkins → New Item

2. Enter:

hiring-app-parallel

3. Select:

Pipeline

4. Click **OK**.

5. Scroll down to **Pipeline**.

6. Select:

Definition: Pipeline script

7. Enter:

node {

```
// =====
```

```
// TOOL CONFIGURATION
```

```
// =====
```

```
def mvnHome = tool(  
    name: 'MVN_HOME',  
    type: 'maven'  
)
```

```
// =====
```

```
// GIT
```

```
// =====
```

```
def GIT_URL = 'https://github.com/betawins/hiring-app.git'
```

```
def GIT_BRANCH = 'main'
```

```
// =====
```

```
// SONARQUBE
```

```
// =====
```

```
def SONAR_SERVER = 'sonarqube-server'
```

```
def SONAR_URL = 'http://52.90.154.40:9002'
```

```
// =====
```

```
// NEXUS

// =====

def NEXUS_URL = '98.93.245.155:8081'
def NEXUS_REPOSITORY = 'hiring_app'
def NEXUS_CREDENTIALS = 'nexus-cred'

def GROUP_ID = 'in.javahome'
def ARTIFACT_ID = 'hiring'
def VERSION = '0.1'

// =====

// TOMCAT

// =====

def TOMCAT_URL = 'http://100.59.202.84:9090'
def TOMCAT_CREDENTIALS = 'tomcat-cred'
def TOMCAT_CONTEXT = 'hiring'

// =====

// SLACK

// =====

def SLACK_CHANNEL = '#all-jenkins-bot'
def SLACK_CREDENTIALS = 'slack-cred'

// =====

// WAR

// =====
```

```
def WAR_FILE = 'target/hiring.war'
```

```
try {
```

```
    // =====
```

```
    // MAVEN PATH
```

```
    // =====
```

```
    env.PATH = "${mvnHome}/bin:${env.PATH}"
```

```
    // =====
```

```
    // STAGE 1 - GIT CLONE
```

```
    // =====
```

```
    stage('Git Clone') {
```

```
        echo ""
```

```
        =====
```

```
        STAGE 1: GIT CLONE
```

```
        =====
```

```
        ""
```

```
        deleteDir()
```

```
        git(
```

```
            branch: GIT_BRANCH,
```

```
    url: GIT_URL
)

sh '''
    set -e

    echo "Git clone completed successfully."

    echo ""
    echo "Current branch:"
    git branch --show-current

    echo ""
    echo "Latest commit:"
    git log -1 --oneline

    echo ""
    echo "Repository files:"
    ls -la
'''
}

// =====
// STAGE 2 - MAVEN BUILD
// =====

stage('Maven Build') {
```

```
    echo ""

    =====

    STAGE 2: MAVEN BUILD

    =====

    ""

    sh ""

        set -e

        echo "Maven version:"

        mvn --version

        echo ""

        echo "Java version:"

        java -version

        echo ""

        echo "Running Maven build..."

        mvn clean package -DskipTests

        echo ""

        echo "Maven build completed successfully."

        echo ""

        echo "Generated WAR:"

        ls -lh target/*.war

    ""

}
```

```
// =====
// STAGE 3 - PARALLEL EXECUTION
// =====

stage('Parallel Quality and Verification') {

    parallel(

        // =====
        // SONARQUBE
        // =====

        'SonarQube Analysis': {

            stage('SonarQube Analysis') {

                echo ""

                =====
                PARALLEL STAGE: SONARQUBE
                =====

                ""

                withSonarQubeEnv(SONAR_SERVER) {

                    sh ""

                    set -e
```

```
echo "Running SonarQube analysis..."
```

```
mvn \\  
org.sonarsource.scanner.maven:sonar-maven-plugin:sonar \\  
-Dsonar.projectKey=hiring-app \\  
-Dsonar.projectName=HiringApp \\  
-Dsonar.projectVersion=${env.BUILD_NUMBER} \\  
-Dsonar.host.url=${SONAR_URL}
```

```
echo ""
```

```
echo "SonarQube analysis completed successfully."
```

```
""
```

```
}
```

```
}
```

```
},
```

```
// =====
```

```
// WAR VERIFICATION
```

```
// =====
```

```
'Verify WAR': {
```

```
stage('Verify WAR') {
```

```
echo ""
```

```
=====
```

```
PARALLEL STAGE: VERIFY WAR
```

```
=====
```

'''

```
sh """"
set -e

echo "Checking WAR file..."

if [ ! -f "${WAR_FILE}" ]; then

    echo "ERROR: WAR file not found."

    echo ""
    echo "Target directory:"
    ls -lah target/

    exit 1
fi

echo ""
echo "WAR file found successfully."

ls -lh "${WAR_FILE}"

echo ""
echo "WAR contents:"

unzip -l "${WAR_FILE}" | head -30

echo ""
```

```

        echo "WAR verification completed."
    """"
}
},

// =====
// APPLICATION INFORMATION
// =====

'Application Information': {

    stage('Application Information') {

        echo ""

        =====
        PARALLEL STAGE: APPLICATION INFORMATION
        =====

        ""

        sh ""

        echo "Application: Hiring App"

        echo ""

        echo "Git branch:"

        git branch --show-current

        echo ""

        echo "Git commit:"

```

```
git log -1 --oneline
```

```
echo ""
```

```
echo "Java version:"
```

```
java -version
```

```
echo ""
```

```
echo "Maven version:"
```

```
mvn --version
```

```
echo ""
```

```
echo "WAR:"
```

```
ls -lh target/hiring.war
```

```
'''
```

```
}
```

```
}
```

```
)
```

```
}
```

```
// =====
```

```
// STAGE 4 - NEXUS UPLOAD
```

```
// =====
```

```
stage('Nexus Upload') {
```

```
    echo ""
```

```
=====
```

```
STAGE 4: NEXUS ARTIFACT UPLOAD
```

=====

'''

```
echo "Nexus URL: ${NEXUS_URL}"
```

```
echo "Repository: ${NEXUS_REPOSITORY}"
```

```
echo "WAR: ${WAR_FILE}"
```

```
nexusArtifactUploader(
```

```
    nexusVersion: 'nexus3',
```

```
    protocol: 'http',
```

```
    nexusUrl: NEXUS_URL,
```

```
    groupId: GROUP_ID,
```

```
    version: VERSION,
```

```
    repository: NEXUS_REPOSITORY,
```

```
    credentialsId: NEXUS_CREDENTIALS,
```

```
    artifacts: [
```

```
        [
```

```
            artifactId: ARTIFACT_ID,
```

```
            classifier: "",
```

```
            file: WAR_FILE,
```

```
            type: 'war'
```

```

    ]
  ]
)

echo ""

=====

NEXUS UPLOAD SUCCESSFUL

=====

'''

}

// =====
// STAGE 5 - TOMCAT DEPLOYMENT
// =====

stage('Tomcat Deployment') {

    echo ""

    =====

    STAGE 5: TOMCAT DEPLOYMENT

    =====

    '''

    echo "Tomcat URL: ${TOMCAT_URL}"
    echo "WAR File: ${WAR_FILE}"
    echo "Context Path: ${TOMCAT_CONTEXT}"

    deploy(

```

```
adapters: [
```

```
    tomcat9(
```

```
        credentialsId: TOMCAT_CREDENTIALS,
```

```
        url: TOMCAT_URL,
```

```
        path: "
```

```
    )
```

```
],
```

```
contextPath: TOMCAT_CONTEXT,
```

```
war: WAR_FILE
```

```
)
```

```
echo ""
```

```
=====
```

```
TOMCAT DEPLOYMENT SUCCESSFUL
```

```
=====
```

```
""
```

```
}
```

```
// =====
```

```

// STAGE 6 - VERIFY TOMCAT

// =====

stage('Verify Tomcat') {

    echo ""

    =====

    STAGE 6: VERIFY TOMCAT

    =====

    ""

    sh ""

    set +e

    echo "Tomcat URL:"
    echo "${TOMCAT_URL}"

    echo ""
    echo "Application URL:"
    echo "${TOMCAT_URL}/${TOMCAT_CONTEXT}/"

    echo ""
    echo "Checking application..."

    HTTP_STATUS=$(curl -s -o /dev/null -w "%{http_code}" \\
"${TOMCAT_URL}/${TOMCAT_CONTEXT}/")

    echo ""
    echo "HTTP Status: ${HTTP_STATUS}"

```

```
if [ "\$HTTP_STATUS" = "200" ]; then
```

```
    echo ""
```

```
    echo "Hiring application is responding successfully."
```

```
elif [ "\$HTTP_STATUS" = "302" ]; then
```

```
    echo ""
```

```
    echo "Hiring application is deployed and returned HTTP 302."
```

```
elif [ "\$HTTP_STATUS" = "404" ]; then
```

```
    echo ""
```

```
    echo "Application returned HTTP 404."
```

```
    echo ""
```

```
    echo "Check the Tomcat deployment and application welcome page."
```

```
else
```

```
    echo ""
```

```
    echo "Application returned HTTP \$HTTP_STATUS."
```

```
fi
```

```
echo ""
```

```
echo "Tomcat verification completed."
```

```
""""
```

```
}
```

```
// =====
```

```
// STAGE 7 - SLACK
```

```
// =====
```

```
stage('Slack Notification') {
```

```
    echo ""
```

```
=====
```

```
STAGE 7: SLACK NOTIFICATION
```

```
=====
```

```
""
```

```
    slackSend(
```

```
        channel: SLACK_CHANNEL,
```

```
        color: 'good',
```

```
        tokenCredentialId: SLACK_CREDENTIALS,
```

```
        botUser: true,
```

```
        message: ""
```

```
PARALLEL PIPELINE SUCCESS
```

Application : Hiring App

Job : \${env.JOB_NAME}
Build : #\${env.BUILD_NUMBER}
Branch : \${GIT_BRANCH}

SonarQube:

\${SONAR_URL}

Nexus:

http://\${NEXUS_URL}

Repository:

\${NEXUS_REPOSITORY}

WAR:

\${WAR_FILE}

Tomcat:

\${TOMCAT_URL}/\${TOMCAT_CONTEXT}/

Jenkins:

\${env.BUILD_URL}

""""

)

}

// =====

// SUCCESS

// =====

```

    echo ""

    =====

    PARALLEL PIPELINE SUCCESS

    =====

    ""

    echo "Application deployed successfully."

    echo ""
    echo "Application URL:"
    echo "${TOMCAT_URL}/${TOMCAT_CONTEXT}/"

}

catch (err) {

    // =====

    // FAILURE

    // =====

    echo ""

    =====

    PARALLEL PIPELINE FAILED

    =====

    ""

    echo "Error: ${err}"

```

```
slackSend(
```

```
    channel: SLACK_CHANNEL,
```

```
    color: 'danger',
```

```
    tokenCredentialId: SLACK_CREDENTIALS,
```

```
    botUser: true,
```

```
    message: ""
```

```
PARALLEL PIPELINE FAILED
```

```
Application : Hiring App
```

```
Job      : ${env.JOB_NAME}
```

```
Build    : #${env.BUILD_NUMBER}
```

```
Branch   : ${GIT_BRANCH}
```

```
Error:
```

```
${err}
```

```
Jenkins:
```

```
${env.BUILD_URL}
```

```
""
```

```
)
```

```
throw err
```

```
}
```

```
finally {
```

```
    echo ""
```

```
=====
```

```
PIPELINE EXECUTION COMPLETED
```

```
=====
```

```
""
```

```
    }
```

```
}
```

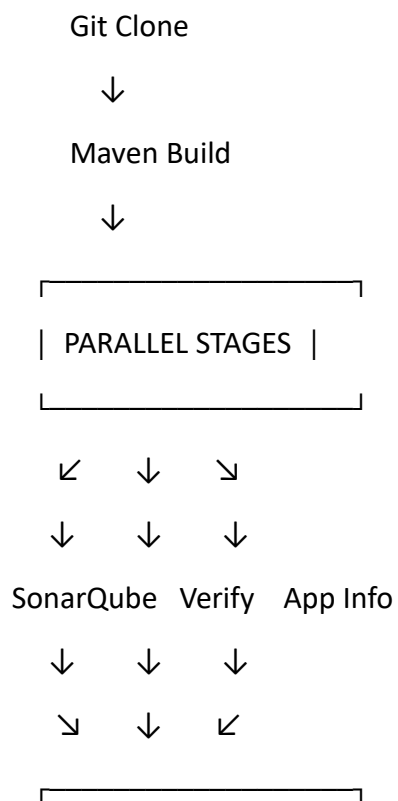
8. Click **Apply**.

9. Click **Save**.

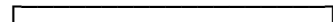
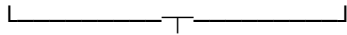
10. Click **Build Now**.

11. Verify the Pipeline stages.

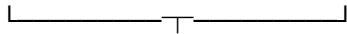
The execution will look like:



| Nexus Upload |



| Tomcat Deployment |



Verify Tomcat



Slack Notification

The screenshot displays the Jenkins web interface for a job named 'hiring_app_parallel'. The browser's address bar shows the URL '3.93.154.157:8080/job/hiring_app_parallel/1/'. The Jenkins page header includes the Jenkins logo, the job name 'hiring_app_parallel', and a dropdown menu showing '#1'. On the left, a sidebar lists various options: Status (selected), Changes, Console Output, Delete build '#1', Git Build Data, Pipeline Overview, Edit build information, Replay, Pipeline Steps, and Workspaces. The main content area shows a green checkmark icon next to the build number '#1 (Oct 6, 2026, 7:35:45 AM)'. Below this, it indicates the build was 'Started by user vamshikumar' and provides the 'Revision' and 'Repository' information. The repository is 'https://github.com/betawins/hiring-app.git' and the specific revision is '9405b107fdffdf68e69fe953ea1f9669784f9d6a'. It also shows the build was started '21 sec ago' and took '16 sec'. A 'git' icon and a 'No changes.' message are also visible. At the bottom right, the text 'REST API' and 'Jenkins 2.580.1' are displayed. The Windows taskbar at the very bottom shows the date '06-10-2026' and time '13:06'.

hiring_app_parallel #1 Console | hiring_app Config - Jenkins | Tools - Manage Jenkins - Jenkins | HTTP Status 404 - Not Found

Not secure 3.93.154.157:8080/job/hiring_app_parallel/1/console

Jenkins / hiring_app_parallel / #1

```
Slack Send Pipeline step running, values are - baseUrl: <empty>, teamDomain: jenkins-bot, channel: #all-jenkins-bot, color: good, botUser: true, tokenCredentialId: slack-cred, notifyCommitters: false, iconEmoji: <empty>, username: <empty>, timestamp: <empty>

[Pipeline] }
[Pipeline] // stage
[Pipeline] echo

=====
PIPELINE SUCCESS
=====

[Pipeline] echo

=====
PIPELINE EXECUTION COMPLETED
=====

[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Not secure 52.90.154.40:9002/projects

Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine.

- There's a new version of SonarQube Server available. Upgrade to SonarQube Server and get access to enterprise features. [Learn More](#)
- The way in which security, reliability, and maintainability counts and ratings are calculated has changed. [Learn more in SonarQube documentation](#)

SonarQube community

Projects | Issues | Rules | Quality profiles | Quality gates | Administration | More

My Favorites | All

Filters

Quality gate

- Passed 3
- Failed 0

Security

Search projects | Perspective | Overall Status | Sort by Name | 3 project(s)

HiringApp Public Passed

Last analysis: 1 hour ago - 45 Lines of Code - XML, Docker, ...

B 2	C 3	A 0	A —	—	0.0%
Security	Reliability	Maintainability	Hotspots Reviewed	Coverage	Duplications

